ARCHITECTURE AND INTERVIEW BRIEF

Pipeline Recovery OS

A self-serve lead recovery product for agents, loan officers, brokers, and local sales teams. It imports messy inactive lead data, scrubs unsafe records, reconstructs lead journeys, and turns recoverable people into a human-approved daily follow-up queue.

Next.js 16

App Router, Server Components, Server Actions, Route Handlers, streaming, and Vercel deployment.

Vercel Workflows

Durable, step-based file ingestion on Fluid Compute. Survives disconnects, refreshes, and restarts — no separate worker service.

Supabase

Postgres source of truth: tenancy + RLS, import job/progress state, staged rows, contacts, opportunities, suppression.

AI SDK + Gateway · Blob

Claude Sonnet journey reconstruction with deterministic fallback; private Vercel Blob raw-file archive.

Problem framing

Audience

Loan officers, mortgage brokerages, real estate teams, and other relationship-driven local sales teams with stale lead data.

Why this matters

Teams already paid for or earned these relationships. The leakage happens after referral, application link, partial application, declined file, or dead deal.

Picture of success

A non-technical assistant can import messy data, accept a cleaned review, and work a focused daily queue without manually maintaining another CRM.

15-minute customer-style demo arc

0 TO 3 MINUTES	**Business context:** "You have leads from texts, Facebook, CRM exports, open houses, and old loan files. The problem is not sales skill. The problem is follow-up memory and process leakage."
3 TO 7 MINUTES	**Import demo:** create a workspace, load exclusions, upload scattered CSV/TXT files, show progress phases, review rows, and accept the import.
7 TO 11 MINUTES	**Daily workbench:** show Today's Follow-Up Queue, a lead work card, suggested message, outcome buttons, and the end-of-day update box.
11 TO 15 MINUTES	**Technical architecture:** show the durable Vercel Workflow processing an import (and surviving a tab close), explain the rendering/caching choices behind sub-second navigation, Supabase for state and tenancy, Blob for private archive, and the AI step with deterministic fallback.

System architecture

The app keeps fast customer-facing product surfaces and slow, messy ingestion on one platform, separated by execution model rather than by vendor. Ingestion is a durable Vercel Workflow, not a request-response route.

User browser

Email sign-in, upload, live progress, review, Today queue, outcomes, reports. Streams a shell instantly via `loading.tsx`.

Vercel Next.js

Route handler archives the upload, creates the job, and triggers the Workflow with `start()`. Server Actions commit accepted rows.

Vercel Workflow

Durable steps on Fluid Compute: parse → normalize → dedupe → suppress → AI reconstruct → stage. Independently retried; resumable.

Supabase

Source of truth: tenancy + RLS, job/progress state, staged `import_rows`, canonical contacts, opportunities, suppression.

Why a Workflow, not a function or a separate worker

Ingestion is long-running and stateful: parse, dedupe, per-group AI reconstruction, and staging. A single request-response function risks the wall-clock timeout and restarts from zero on any failure. Vercel Workflows give durable, step-level execution — each step is independently retried and the run survives deploys, instance restarts, and client disconnects. Progress lives in Postgres, so the browser can close and reopen mid-import and resume against live state. This is the assessment's "leverage Workflows for stateful applications" and "persist state through disconnections" made concrete.

Production deployment

Vercel app + workflow

https://lead-recovery-os.vercel.app

Project: `lead-recovery-os`. The app and the durable import Workflow deploy together — one project, no separate service.

Supabase

Source of truth: auth, RLS, import job/progress state, staged rows, canonical contacts, opportunities, and suppression records.

Vercel Blob

Private raw-file archive for audit and reprocessing, with a Supabase Storage fallback when Blob is not configured.

The production proof path is a live import: Vercel archives the file, creates a Supabase job, and triggers the Workflow with `start()`; the workflow's steps advance progress in Postgres while the browser polls; the user then accepts staged rows into the Today queue. Closing the tab mid-import and reopening it resumes against live progress — nothing is lost.

Data flow

1. SIGNUP	User signs up passwordlessly through Supabase Auth and creates a workspace. The workspace chooses a pipeline template such as `mortgage_growth`.
2. UPLOAD	Next.js receives a file or pasted text from `/imports/new` through `/api/imports`.
3. ARCHIVE	The raw input is stored privately in Vercel Blob when configured, with fallback metadata if Blob credentials are unavailable locally.
4. TRIGGER WORKFLOW	Next.js creates `imports` and `import_jobs` rows, then triggers the durable Workflow with `start()`. The call returns immediately; the browser moves to the processing screen.
5. PARSE STEP	The first Workflow step downloads the archived file and parses CSV, TSV, XLSX, TXT, JSON, VCF, DOCX, and text-based PDF (via `unpdf`). Legacy binary XLS is intentionally rejected with guidance to export as XLSX/CSV/TSV.
6. NORMALIZE	Names, phones, emails, notes, consent status, sources, dates, and lead stages are normalized.

7. DEDUPE	Email and phone identity keys match existing contacts. Repeated records are grouped into one staged journey.
8. SUPPRESS	DNC, opt-out, closed, funded elsewhere, bad contact, and existing exclusions are held out before drafts are created.
9. AI RECONSTRUCT	AI SDK + Vercel AI Gateway reconstruct messy journeys for ambiguous groups. A deterministic fallback runs when AI is not configured or fails — the eval suite enforces the same stage/hallucination contract on both paths.
10. STAGE + PROGRESS	The step writes staged `import_rows` and advances `workflow_percent` in Supabase throughout. Because progress is persisted, the browser survives refresh, disconnect, and instance restart.
11. REVIEW	The user sees Ready, Needs review, and Suppressed rows. A red banner makes clear nothing is in their queue yet.
12. ACCEPT	The accept Server Action writes contacts, opportunities, drafts, identities, journey events, and exclusions into canonical tables. This is the human commit boundary.

Vercel features used

FEATURE	WHERE IT APPEARS	WHY IT IS APPROPRIATE NOW
Next.js App Router	`src/app` routes for landing, demo, signup, onboarding, dashboard, imports, leads, reports, and exports.	Lets server-render authenticated product surfaces while keeping public demo pages static where possible.
React Server Components	Dashboard, imports, reports, leads, settings.	Data-heavy views can query Supabase server-side without shipping unnecessary client code.
Server Actions	Workspace creation, accepting staged imports, outcome buttons, daily dump updates, draft edits.	High-leverage form mutations stay close to the UI with simple progressive forms and revalidation.
Route Handlers	`/api/imports`, `/api/imports/[id]/status`, `/api/exports/approved`, auth callbacks.	They provide bounded API edges for upload, polling, export, and auth.
Vercel Workflows	`src/workflows/import-workflow.ts` (`use workflow` / `use step`); triggered by `start()` in the upload route.	Durable, step-based ingestion that survives disconnects and restarts. The headline reason Vercel fits this use case — no separate worker, no wall-clock ceiling, step-level retries.
Fluid Compute	Runtime for the workflow steps (full Node.js parsers: papaparse, read-excel-file, mammoth, unpdf).	Full Node with warm instances and a long timeout — heavy file parsing runs without a container.
	`dashboard`, `leads`, `queue` loading skeletons;	The shell paints instantly while per-user data

FEATURE	WHERE IT APPEARS	WHY IT IS APPROPRIATE NOW
Streaming (loading.tsx)	root `error.tsx` / `not-found.tsx`.	resolves, improving perceived load and protecting the demo from a blank crash.
Vercel Blob	Private raw import archive through `src/lib/import-archive.ts`.	Stores original files for audit and reprocessing without using Postgres as file storage.
AI SDK	Import journey reconstruction (in the workflow step), daily dump parsing, and draft refinement.	Provides one model interface, structured outputs, prompt safety, and consistent fallback behavior.
Vercel AI Gateway	Model routing for `anthropic/claude-sonnet-4.6` and `openai/gpt-5.5` paths when configured.	Centralizes provider access and model choice while preserving deterministic fallback if not configured.
Web Analytics and Speed Insights	Root layout instrumentation.	Useful for public demo traffic and customer-facing performance without third-party analytics complexity.
OpenTelemetry	`src/instrumentation.ts` and `src/lib/tracing.ts`.	Traces workflow events without logging PII such as names, emails, phones, or message text.
Preview deployments	Release workflow, not a code path.	Lets product changes be tested against a preview URL before production is promoted.

Why Vercel Workflows for stateful ingestion

Not every job is a request-response function

Parsing arbitrary customer files, deduping, and per-group AI reconstruction is long-running and multi-phase. A single function risks the wall-clock timeout and restarts from zero on any failure. I considered an external container (Railway) — it's in the git history as the evaluated alternative — but it adds a second platform, a polling loop running 24/7, and its own deploy story.

Workflows give durability without a second platform

The Workflow DevKit runs each phase as an independently retried `use step` on Fluid Compute, orchestrated by a `use workflow` function. Runs survive deploys, instance restarts, and client disconnects. One project, one deploy, native observability — and the "when not to use a plain function" judgment the role asks for.

The key design choice is that the workflow never writes active contacts directly — it only stages rows in `import_rows`. The Vercel app commits accepted rows after human review. State of record stays in Supabase (queryable, RLS-protected, drives the review UI), not in workflow storage. **Scale path:** for very large files the per-group AI loop becomes batched sub-steps for finer retry granularity, and Vercel Queues can buffer bursts ahead of the workflow.

Rendering & performance (Track A)

The authenticated surfaces are genuinely per-user, per-organization, real-time data — there is no shared, cacheable shell to prerender, so the win is not caching this data (on serverless, in-memory `use cache` doesn't persist across requests anyway). The win is **removing redundant round-trips and streaming the shell**. Honest rendering choices beat cargo-culted caching.

Before

Clicking an outcome button took 2–3s. Each navigation made two Supabase `auth.getUser()` network calls (middleware + page) and ran a duplicate membership query, all serial and uncached, then repeated after every mutation.

After (sub-second)

Sessions verify locally with `getClaims()` (cached JWKS) instead of an Auth-server round-trip; the middleware matcher excludes API and asset paths; `React.cache()` dedupes per-render queries; and `loading.tsx` streams the shell instantly.

Static vs dynamic

Marketing, login, and signup are static and CDN-cached. App routes are dynamic by necessity (per-user data) — used deliberately, not by default.

Core Web Vitals

Streaming a skeleton improves perceived LCP; eliminating the auth waterfall improves INP on every interaction; stable skeleton geometry avoids layout shift (CLS).

Cost

Fewer Auth-server calls and deduped queries mean fewer function invocations and less Active CPU per navigation — performance and cost move together.

AI design

Prompting

The import prompt only sees normalized lead data and raw source rows for that grouped person. It is instructed not to invent names, dates, outcomes, or recommendations for suppressed records.

Context selection

The system parses and dedupes first. AI is reserved for duplicate groups, long notes, Facebook/ iMessage-style text, and ambiguous journey summaries.

Fallback behavior

If `AI_GATEWAY_API_KEY` is missing or the model fails, the workflow step uses deterministic reconstruction. Imports continue with lower confidence rather than failing the user.

Model choice

`anthropic/claude-sonnet-4.6` is used for messy import journey reconstruction because the task is language-heavy and benefits from stronger reasoning. The app keeps token cost controlled by preprocessing before AI.

Cost and latency trade-off

Simple rows do not need model calls. AI is selective, bounded by row grouping and note length. The browser sees progress from Supabase while the workflow processes asynchronously.

Lightweight evaluation approach

The project includes a small but defensible regression set rather than a large, murky eval suite.

Parser fixtures

`evals/fixtures/` verifies CSV, repeat import, suppression, JSON, VCF, and messy notes parsing against the live `src/lib/import-pipeline` modules.

AI journey regression

`evals/import-journey-regression.json` verifies stage classification, preserved context, and no invented outcomes — a hallucination guard on the AI step's fallback contract.

Human E2E QA

`qa/self-serve-import-flow.mjs` creates a fresh user, imports demo files, accepts rows, tests outcome buttons and the daily dump, and verifies persistence.

One command runs both eval checks against the same code the production workflow uses, so the eval and the pipeline can't drift. It prints a readable PASS/FAIL table and exits non-zero on failure, so it can gate CI.

```
npm run eval            # parser fixtures + journey/hallucination
regression
npm run eval:parsers    # format coverage only
npm run eval:ai         # journey / hallucination regression only

npm run lint && npm run build
APP_URL=https://lead-recovery-os.vercel.app npm run qa:selfserve-
import
```

Security, privacy, and reliability

Tenant isolation

Supabase RLS ensures users only see organizations where they are members. Service-role writes are limited to server-side route handlers, Server Actions, and workflow steps.

No outbound sending

V1 generates drafts and exports queues. It does not send SMS, email, iMessage, Facebook messages, or referral-compensation language.

Suppression first

DNC, opt-out, closed, funded elsewhere, bad contact, and no-consent records are held out before drafts are created.

Human commit boundary

The workflow stages data. The user accepts before records become active contacts and opportunities.

Private raw archive

Original imports are archived privately for audit and reprocessing. Clients do not receive public raw-file URLs.

PII-safe traces

Workflow telemetry avoids names, emails, phones, note bodies, and draft text.

Production boundaries and trade-offs

DECISION	WHY	DEFERRED ALTERNATIVE
Polling Supabase for import progress	Survives browser refresh, instance restart, and preview/prod boundaries — progress is persisted, not held in memory.	Supabase Realtime or SSE for push-based progress when volume and UX justify it.
Durable Vercel Workflow for ingestion	Long-running, multi-phase, stateful work with step-level retries and disconnection survival — on one platform.	Batched AI sub-steps and Vercel Queues for burst buffering at higher volume; Sandbox to isolate untrusted file parsing.
No automated outbound messaging	Compliance and trust. The product is a human-approved recovery layer.	Future integrations with explicit consent, audit logs, and broker controls.
No heavy CI/CD scaffold yet	The interview and sales need a small, deep, defensible product now.	Broader CI, load tests, and integration environments after pilot traction.
Legacy XLS not supported	Binary XLS parsers introduce avoidable dependency risk. XLSX/CSV/TSV are acceptable in V1.	Enterprise-specific parser support after a customer requires it.

Enterprise customer pitch

"This is a safe recovery layer, not a CRM replacement. Your team imports stale leads, old applications, referrals, dead deals, and suppression lists. The app archives the raw data privately, processes it in a durable workflow that survives disconnects, stages clean rows for human review, and creates a daily queue only after acceptance. AI helps summarize messy journeys, but deterministic rules own suppression, dedupe, and final writes. That means the business value is recovered conversations, while the technical posture stays auditable and human-supervised."

What I would say if asked "why these platform choices?"

Why Vercel?

It is the best product surface for a Next.js app: fast preview deployments, App Router fluency, analytics, speed insights, Blob, AI SDK/Gateway, and OpenTelemetry in the same operational ecosystem.

Why Supabase?

Auth, Postgres, RLS, and Storage-adjacent patterns make it fast to ship a multi-tenant MVP without building identity or permissions from scratch.

Why Vercel Workflows (and not a container)?

Ingestion needs durability and state, not just compute. Workflows give resumable, step-level execution that survives disconnects — on the same platform as the product surface, with no second deploy target. I evaluated an external container and chose Workflows on merits.

Why not build every Vercel feature?

Platform understanding includes restraint. Vercel Sandbox (untrusted-file hardening), Queues (burst buffering), and finer AI batching are the next steps, called out where they'd earn their place — but the current product needs imports, review, daily work, auditability, and proof of value.

Code map

AREA	PATH	PURPOSE
Signup and onboarding	`src/app/signup`, `src/app/onboarding`	New users enter the product and create a workspace.
Import route	`src/app/api/imports/route.ts`	Accepts upload/paste, archives raw data, creates job rows, triggers the workflow with `start()`.
Import workflow	`src/workflows/import-workflow.ts`	`use workflow` orchestrator + two `use step` phases (parse, analyze/AI/stage).
Pipeline logic	`src/lib/import-pipeline/`	Parsers, normalize, dedupe, suppress, AI reconstruct, stage — full Node, runs inside the steps.
Import progress	`src/components/import-progress.tsx`	Polls status and displays workflow phases; resumes against persisted progress.
Accept import	`src/app/actions.ts`	Commits staged rows into canonical records after human review.
Auth + provisioning	`src/app/auth/demo-login/route.ts`, `src/lib/provision.ts`	Email sign-in auto-provisions an isolated, seeded workspace; returning emails resolve to the same one.
Perf layer	`src/lib/supabase-server.ts`, `src/lib/data.ts`, `src/lib/middleware.ts`	Local `getClaims()` session verification + `React.cache()` de-duplication.
Eval suite	`evals/run.mts`	`npm run eval`: parser fixtures + journey/hallucination

AREA	PATH	PURPOSE
		regression against the live pipeline.

Interview close

"The point of this build is not that I used every feature available. The point is that I drew the boundaries carefully. Vercel owns the product surface, the durable ingestion workflow, the AI platform, observability, and previews. Supabase owns tenancy, RLS, and source-of-truth state. AI is useful where messy human notes need interpretation, but it is not trusted with suppression, final writes, or outbound communication. And the performance work is honest: I removed round-trips and streamed the shell rather than caching data that is genuinely per-user. The result is a product that is sellable to a broker today and architecturally honest enough to defend in an enterprise conversation."